

Documentazione tecnica

Team SalentEasy

Il notebook implementa una pipeline completa di Machine Learning per classificare la razza di un gatto a partire da caratteristiche fisiche e comportamentali. L'obiettivo non è soltanto ottenere previsioni accurate, ma costruire un flusso di lavoro ordinato, riproducibile e facilmente comprensibile.

Librerie

Pandas viene utilizzata per leggere e manipolare i dataset CSV tramite DataFrame, struttura che consente filtraggio, statistiche e trasformazioni. NumPy fornisce operazioni numeriche efficienti. Matplotlib e Seaborn sono impiegate per realizzare i grafici dell'EDA e della matrice di confusione. Scikit-learn costituisce il cuore del progetto: contiene gli algoritmi di preprocessing, suddivisione del dataset, addestramento, ricerca degli iperparametri e valutazione.

Caricamento dati

Prima dell'addestramento il codice verifica l'esistenza dei file richiesti. Questa scelta evita errori durante l'esecuzione e rende il notebook più robusto. Successivamente viene effettuata una pulizia della variabile target eliminando eventuali etichette mancanti o non valide, poiché un algoritmo supervisionato necessita sempre della risposta corretta durante la fase di apprendimento.

EDA

L'Analisi Esplorativa serve a comprendere il dataset prima di costruire il modello. Vengono analizzate la distribuzione delle razze, le correlazioni tra le variabili numeriche, la relazione tra peso e razza e successivamente la matrice di confusione. L'EDA permette di individuare squilibri, anomalie e possibili relazioni che possono influenzare l'accuratezza del modello.

Preprocessing

Le variabili categoriche devono essere convertite in rappresentazioni numeriche perché gli algoritmi di Machine Learning operano esclusivamente su numeri. Il preprocessing rende inoltre i dati coerenti tra training e test, evitando differenze di codifica. Train/Test split. Il dataset viene suddiviso in una parte utilizzata per apprendere e una parte utilizzata esclusivamente per valutare il modello. Questa separazione permette di stimare la capacità di generalizzazione ed evita di valutare il modello sugli stessi dati con cui è stato addestrato.

Random Forest

Il modello scelto è una Random Forest. La decisione è motivata dal fatto che questo algoritmo offre un ottimo compromesso tra semplicità, accuratezza e robustezza. Una Random Forest è composta da molti alberi decisionali costruiti su campioni differenti del dataset (bootstrap). Ogni albero esprime una previsione e la decisione finale deriva dal voto della maggioranza. Questo riduce l'overfitting tipico dei singoli Decision Tree e rende il modello stabile anche in presenza di dati rumorosi. Inoltre gestisce contemporaneamente variabili numeriche e categoriche codificate, richiede poche assunzioni statistiche ed è adatto a dataset di dimensioni come quello della challenge.

Ricerca degli iperparametri

Il notebook utilizza GridSearchCV per provare automaticamente diverse configurazioni della Random Forest. In questo modo non viene scelto un insieme di parametri arbitrario, ma viene individuata la configurazione che produce i risultati migliori mediante cross validation. La cross validation ripete più addestramenti su differenti suddivisioni del dataset, riducendo il rischio che una singola divisione influenzi il risultato.

Valutazione

Oltre all'accuracy vengono analizzate metriche come F1-score e matrice di confusione. L'accuracy misura la percentuale di previsioni corrette, mentre l'F1-score considera contemporaneamente precision e recall risultando più affidabile quando le classi non sono perfettamente bilanciate. La matrice di confusione mostra per ogni razza quali esempi vengono classificati correttamente e quali vengono confusi con altre categorie.

Predizione finale

Dopo aver selezionato il modello migliore, esso viene addestrato utilizzando il training set e applicato al test set privo della colonna 'razza'. Le predizioni vengono salvate nel file predictions.csv nel formato richiesto dalla challenge (ID e razza_prevista). Separare il file di output dal notebook garantisce riproducibilità e facilita la consegna.

Considerazioni progettuali

L'organizzazione del notebook segue una pipeline lineare: verifica dei file, caricamento, analisi, preprocessing, addestramento, ottimizzazione, valutazione e generazione dell'output. Questa struttura migliora la leggibilità del codice, semplifica il debugging e rende ogni fase indipendente e facilmente documentabile. Le numerose sezioni commentate presenti nel notebook riflettono la volontà di spiegare non solo cosa viene eseguito, ma anche il motivo tecnico di ogni scelta.